ENTERPRISE SECURITY ASSESSMENT LAB

API LAB DEPLOYMENT AND TARGET AVAILABILITY VERIFICATION

Docker container validation, crAPI browser reachability, and safe target readiness confirmation

Field	Details
Module	Sub-Project 2 - API Security Testing Lab
Document Type	Individual API Security Methodology PDF
Phase / Test Case	Phase 1 - API Lab Deployment
Prepared For	GitHub Portfolio Documentation
Prepared By	Jagriti Banerjee
Repository	Enterprise-Security-Assessment-Lab
GitHub Filename	01-api-lab-deployment-methodology-report.pdf

This document is a professional, evidence-driven explanation of one API security methodology section. It is designed for portfolio review and contains only authorized lab-based testing context. Sensitive token values, account data, cookies, and private identifiers are redacted or intentionally represented with placeholders.

1. Section Overview

Area	Details
Phase	Phase 1 - API Lab Deployment
Objective	Confirm that the API lab environment is running, reachable, and safe to test before any API security assessment begins.
Primary Result	Evidence captured and methodology documented
GitHub Folder	02-api-security/reports/methodology/

Why this section matters

The deployment evidence proves that the API target was self-hosted and intentionally vulnerable, which is important for ethical testing. Before API security testing begins, the tester must prove that the target exists, is reachable, and belongs to the authorized lab scope.

2. Evidence Embedded in This PDF

Evidence 1: 00-crapi-docker-containers-running.png

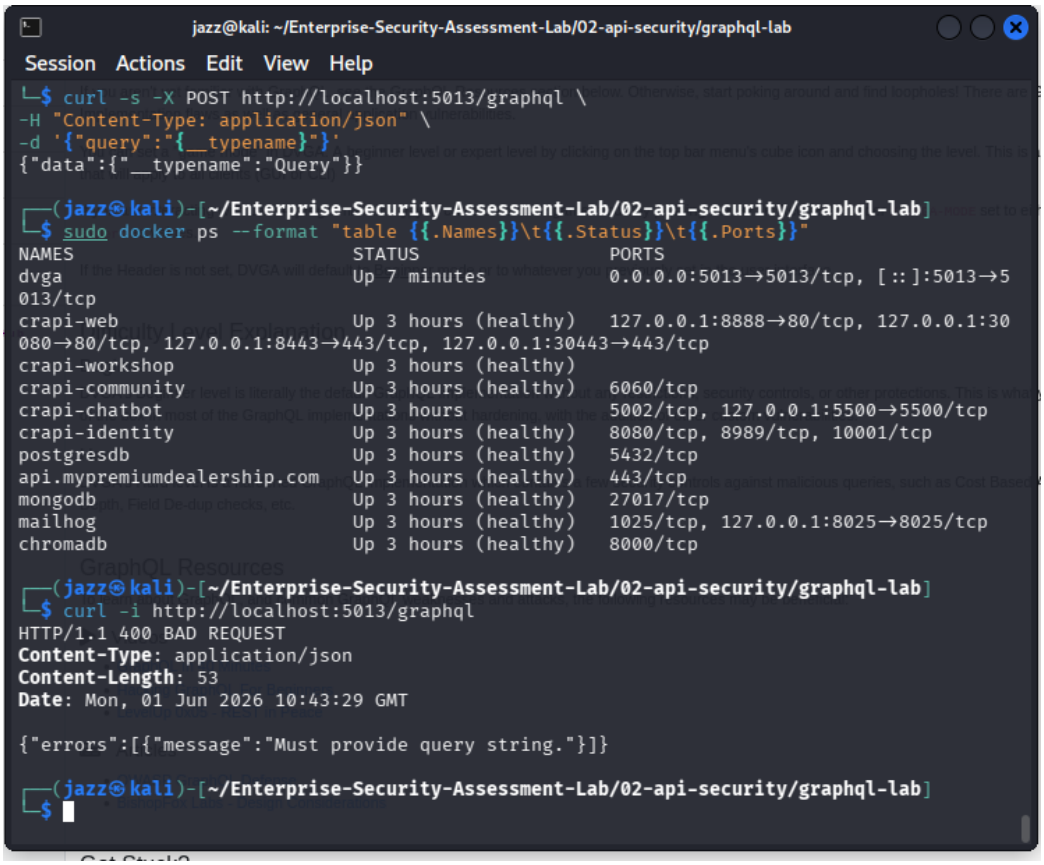

Figure 1 - Docker container evidence showing the crAPI lab services required for the API target.

Docker container evidence showing the crAPI lab services required for the API target.

Evidence 2: 01-crapi-homepage-browser.png

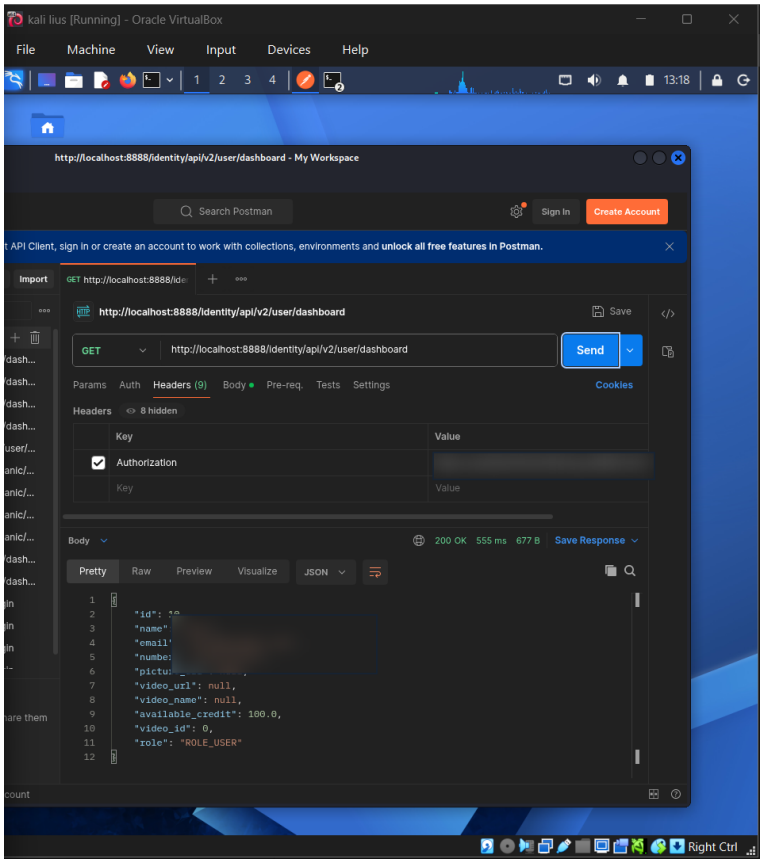

Figure 2 - Browser evidence confirming the crAPI web interface is reachable at the local lab URL.

Browser evidence confirming the crAPI web interface is reachable at the local lab URL.

3. Command and Workflow Used

```
# Check running Docker containers
docker ps

# Confirm crAPI is reachable in browser
http://127.0.0.1:8888

# Optional service availability check
curl -I http://127.0.0.1:8888
```

Step-by-step workflow

- Started or verified the crAPI lab stack through Docker.
- Reviewed running containers to ensure required services were active.
- Opened the lab application in the browser to confirm availability.
- Confirmed the assessment remained within the local, authorized lab environment before continuing.

4. Professional Interpretation

The deployment evidence proves that the API target was self-hosted and intentionally vulnerable, which is important for ethical testing. Before API security testing begins, the tester

must prove that the target exists, is reachable, and belongs to the authorized lab scope.

5. Security Risk and Impact

This phase is not a vulnerability finding. Its risk value is operational: testing an incorrect or unauthorized target would invalidate the assessment and could create legal or ethical issues. Professional API testing always starts with scope and target validation.

6. Recommended Remediation and Hardening

- Use a dedicated local lab or isolated network for vulnerable API testing.
- Document the exact target URL and container state before beginning testing.
- Stop or destroy lab containers after the assessment to reduce unnecessary exposure.
- Avoid exposing vulnerable API labs to public networks.

7. Framework Mapping

Framework Area	Mapping
OWASP API	Assessment preparation / safe lab validation
PTES	Pre-engagement and intelligence preparation
Evidence Result	Lab deployment proof and target availability proof

8. Sanitized Output Style for GitHub

When documenting this evidence publicly, use placeholders instead of live values. The goal is to prove methodology and security understanding without exposing secrets or private data.

```
Authorization: Bearer <REDACTED_TOKEN>
Cookie: <REDACTED_COOKIE>
email: lab-user@example.com
password: REDACTED
object_id: <CONTROLLED_LAB_ID>
redirect_uri: <REDACTED_OR_LAB_CALLBACK>
```

9. GitHub Redaction Checklist

- Bearer tokens and JWT values must be blurred or replaced with <REDACTED_TOKEN>.
- Email addresses, phone numbers, user IDs, and object identifiers should be blurred when they are not required for understanding the evidence.
- Authorization headers, cookies, session values, OAuth codes, refresh tokens, access tokens, and client secrets must never be visible in public GitHub screenshots.
- If a screenshot contains personally identifiable data, crop the view or blur the affected rows before publication.

10. Publication Note

This report is designed for GitHub portfolio use. It describes authorized lab testing only and avoids production claims. The embedded screenshots have been treated as lab evidence and should remain redacted before upload.